SquatCheck — Architecture & Implementation Plan

HW10 Technical Prototype · Jolie, Sofia, Sam

1. Overview

This doc is the shared contract for HW10. If everyone builds to what's written here, the three pieces (data, UI, click-through) will plug together. When in doubt, follow this doc over memory — update the doc if we decide to change something.

Stack: Flask backend · HTML + JS + jQuery + Bootstrap frontend · JSON for lesson/quiz data.

Scope: Single user at a time. No accounts. Store selections server-side in a simple in-memory dict (reset on server restart is fine for HW10).

How to use the prompts in this doc: Each section ends with a Claude Code prompt (green box). Open Claude Code in the repo directory, paste the prompt for the piece you own, and let it draft the file. Read what it writes before committing — if something looks off, push back on it or ask it to revise. Prompts are a starting point, not final code.

2. Job Assignments

Person	Role	Owns
Sam	Data architect (both parts)	app.py routes, lessons.json + quiz.json, session storage, /submit-answer logic, scoring.
Jolie	UI implementer (both parts)	Jinja templates (home, learn, quiz, result), Bootstrap layout, static CSS, translating prototype slides into HTML.
Sofia	Tester + click-through (both parts)	End-to-end manual test script, running through every state, filing issues on GitHub, verifying backend receives data.

Everyone commits to the repo. HW10 requirement: every team member must check something in. If one role finishes early, help the person blocking the critical path (usually UI or integration).

3. Repo Structure

Create this on day one so nobody is guessing where files go:

```
squatcheck/
├── app.py                      # Sam — Flask routes
├── requirements.txt            # Sam — flask
├── data/
│   ├── lessons.json            # Sam — learning content
│   └── quiz.json               # Sam — quiz questions + answers
├── static/
│   ├── css/style.css           # Jolie
```

```
│   ├── js/app.js               # Jolie (optional, for jQuery handlers)
│   └── img/                    # prototype images (heel_good.png, etc.)
├── templates/
│   ├── base.html               # Jolie — Bootstrap scaffold, nav
│   ├── home.html               # Jolie
│   ├── learn.html              # Jolie (one template, data-driven)
│   ├── quiz.html               # Jolie (one template, data-driven)
│   └── result.html             # Jolie
├── tests/
│   └── clickthrough_checklist.md  # Sofia
└── README.md
```

Branch policy: everyone works on a branch named after themselves (sam/, jolie/, sofia/). Merge to main via PR once your piece works. Don't push broken code to main.

▸ **Claude Code prompt — Scaffold the repo (anyone, run first)**

```
In the current empty directory, create this exact structure for a Flask app called
SquatCheck:

squatcheck/
├── app.py (just a hello-world Flask app on port 5000 for now)
├── requirements.txt (flask only)
├── data/lessons.json (empty array: {"lessons": []})
├── data/quiz.json (empty array: {"questions": []})
├── static/css/style.css (empty)
├── static/js/app.js (empty)
├── static/img/ (empty dir, add .gitkeep)
├── templates/base.html (minimal Bootstrap 5 CDN scaffold with a content block)
├── templates/home.html, learn.html, quiz.html, result.html (all extend base.html, empty
content block)
├── tests/clickthrough_checklist.md (empty)
└── README.md (just the project name and how to run: pip install -r requirements.txt,
then flask --app app run)

Also create a .gitignore with __pycache__/, *.pyc, .venv/, .env.
Do not add any extra files or features. Keep it minimal — this is a skeleton for three
people to fill in.
```

4. Data Architecture — Sam

4.1 Flask Routes

Route	Method	Purpose
/	GET	Render home.html. Reset session state on visit so every new run starts clean.
/start	POST	Record start timestamp for the user, then redirect to /learn/1.

Route	Method	Purpose
/learn/<int:n>	GET	Render learn.html with the nth lesson from lessons.json. Log time entered.
/quiz/<int:n>	GET	Render quiz.html with the nth question from quiz.json.
/submit-answer	POST	Accept JSON {question_id, answer}. Store in session. Return {correct: bool, next_url: str}.
/result	GET	Compute score from stored answers, render result.html with score + per-question breakdown.

4.2 lessons.json — shape

One array of lesson objects. The learn template reads whichever index matches /learn/<n>.

```
{
  "lessons": [
    {
      "id": 1,
      "title": "What is a Compensation Pattern?",
      "type": "intro",
      "body": "When someone squats, their body sometimes works around...",
      "bullets": [
        { "label": "Heel Rise", "desc": "Heels lift off the ground..." },
        { "label": "Excessive Forward Lean", "desc": "..." },
        { "label": "Lumbar Flexion", "desc": "..." }
      ],
      "image": null
    },
    {
      "id": 2,
      "title": "Heel Position",
      "type": "comparison",
      "good": { "image": "heel_good.png", "caption": "Heels flat on ground" },
      "bad":  { "image": "heel_bad.png",  "caption": "Heels lifted off ground" },
      "tip":  "Watch the ankles. If heels come up during the squat, ..."
    }
  ]
}
```

Lesson plan (5 lessons): (1) Intro · (2) 3 Things to Watch · (3) Heel Position · (4) Torso-Tibia Alignment · (5) Lumbar Flexion · (6) Review. Matches the prototype slides.

4.3 quiz.json — shape

Three questions. Include the answer key server-side so it's never exposed in the rendered HTML.

```
{
  "questions": [
    {
      "id": 1,
      "type": "multi_select",
      "prompt": "Watch this side-view squat. Do you see any compensation patterns?",
      "image": "quiz_q1.png",
      "options": ["Heel Rise", "Excessive Forward Lean", "Lumbar Flexion", "No
compensations"],
      "correct": ["Excessive Forward Lean", "Lumbar Flexion"],
      "explanation": "Heels stay flat (pass). Torso leans way past shin. Lower back
rounds at bottom."
    },
    {
      "id": 2,
      "type": "single_select",
      "prompt": "Read this observer's notes. Which compensation pattern?",
      "notes": "From the side, I can see her torso is angled way further forward than
her shin...",
      "options": ["Heel Rise", "Excessive Forward Lean", "Lumbar Flexion"],
      "correct": "Excessive Forward Lean",
      "explanation": "The key clue is torso angled further than shin..."
    },
    {
      "id": 3,
      "type": "true_false_multi",
      "prompt": "True or False? Mark each statement.",
      "statements": [
        { "text": "If heels lift, it means knees are weak.", "correct": false },
        { "text": "Lumbar flexion = lower back rounds at bottom.", "correct": true },
        { "text": "Torso-tibia alignment = compare torso to shin.", "correct": true
},
        { "text": "Knee valgus is best assessed from the side.", "correct": false }
      ]
    }
  ]
}
```

4.4 Session storage

Simplest thing that works for one user. Use Flask's `session` or a module-level dict. Shape:

```
user_state = {
  "start_time": "2026-04-20T14:03:00",
  "lesson_visits": { 1: "14:03:05", 2: "14:04:12", ... },
  "answers": {
    1: { "answer": ["Excessive Forward Lean", "Lumbar Flexion"], "correct": true },
    2: { "answer": "Excessive Forward Lean", "correct": true },
    3: { "answer": [false, true, true, false], "correct": true }
  }
}
```

Scoring: count questions where `correct == true`. Display as N/3 on /result.

▸ Claude Code prompt — Sam — build app.py routes

```
In app.py, build a Flask app with these routes. Assume single user, no auth. Use Flask's
session for state, with a secret_key.


Routes:
  GET  /                  — render home.html. Clear session on load so a fresh run starts
clean.
  POST /start             — set session['start_time'] = now ISO string. Redirect to
/learn/1.
  GET  /learn/<int:n>     — load data/lessons.json, pick lessons[n-1], record
session['lesson_visits'][n] = now. Render learn.html with the lesson object and total
lesson count. 404 if n out of range.
  GET  /quiz/<int:n>      — load data/quiz.json, pick questions[n-1]. Render quiz.html
with the question and total question count. Do NOT send the correct answer to the
template.
  POST /submit-answer     — accept JSON {question_id, answer}. Load quiz.json, check
correctness per question type (multi_select: set equality; single_select: string
equality; true_false_multi: list-of-bools equality). Store in
session['answers'][question_id] = {answer, correct}. Return JSON {correct: bool,
next_url: str} where next_url is /quiz/{n+1} or /result if last.
  GET  /result            — compute score from session['answers'], render result.html
with score, total, and per-question breakdown.


Use JSON files via json.load, no DB. Print session state to console on every request so
Sofia can verify data storage. Keep it under ~120 lines.
```

▸ Claude Code prompt — Sam — fill in lessons.json

```
Fill data/lessons.json with 5 lessons matching this structure. Content should match the
SquatCheck prototype about reading a side-view squat assessment.


Lesson 1 (type: intro): "What is a Compensation Pattern?" — body explains compensation
patterns, followed by 3 bullets naming Heel Rise, Excessive Forward Lean, Lumbar Flexion
with one-line descriptions each.
Lesson 2 (type: intro): "3 Things to Watch (Side View)" — body lists the 3 checks: heel
position, torso-tibia alignment, lumbar curve, each with what to watch and what it means
if compensated.
Lesson 3 (type: comparison): "Heel Position" — good={image:'heel_good.png',
caption:'Heels flat on ground'}, bad={image:'heel_bad.png', caption:'Heels lifted off
ground'}, tip explains limited ankle mobility.
```

```
Lesson 4 (type: comparison): "Torso-Tibia Alignment" — good/bad images lean_good.png / lean_bad.png, tip about tight hips or weak core.
Lesson 5 (type: comparison): "Lumbar Flexion Control" — good/bad images lumbar_good.png / lumbar_bad.png, tip about limited hip mobility / butt wink. IMPORTANT: include an extra field "side_by_side_note" that explains the difference between forward lean (whole torso tilts) and lumbar flexion (just low back rounds). This came from user testing.


Every lesson has id, title, type, and type-specific fields. Keep language plain — assume a reader with no gym background (user test found 'dorsiflexion' was too jargony).
```

▸ Claude Code prompt — Sam — fill in quiz.json

```
Fill data/quiz.json with 3 questions matching this schema. Content based on the SquatCheck prototype.


Q1 (type: multi_select): prompt "Watch this side-view squat. Do you see any compensation patterns?", image: 'quiz_q1.png', options: ["Heel Rise", "Excessive Forward Lean", "Lumbar Flexion", "No compensations"], correct: ["Excessive Forward Lean", "Lumbar Flexion"], explanation about heels passing but torso and low back failing.


Q2 (type: single_select): prompt "Read this observer's notes. Which compensation pattern are they describing?", notes field with text about torso angled way further forward than shin, heels flat, lower back fine. options: ["Heel Rise", "Excessive Forward Lean", "Lumbar Flexion"], correct: "Excessive Forward Lean", explanation points out the torso-vs-shin clue.


Q3 (type: true_false_multi): prompt "True or False? Mark each statement.", statements array of 4: (1) 'If heels lift, it means knees are weak' → false, (2) 'Lumbar flexion = lower back rounds at bottom' → true, (3) 'Torso-tibia alignment = compare torso to shin' → true, (4) 'Knee valgus is best assessed from the side' → false.


Every correct answer and explanation lives in this file — the template never sees it.
```

5. UI Implementation — Jolie

5.1 base.html

Bootstrap 5 CDN. Green header bar matching the prototype (#2E5E4E). Back/Next nav at the bottom as a reusable block. Every page extends this.

5.2 Templates and what they render

Template	Renders
home.html	SquatCheck title, subtitle, big green "Start Learning" button that POSTs to /start.
learn.html	Reads lesson object. If type=intro: title + body + numbered list. If type=comparison: two side-by-side cards (green header / red header) with image + caption, plus tip below. Next button links to /learn/{n+1}, or /quiz/1 if last lesson.
quiz.html	Reads question object. Switch on type: multi_select renders checkboxes, single_select renders radio buttons, true_false_multi renders T/F pair per

Template	Renders
	statement. Submit button posts to /submit-answer via jQuery AJAX, then advances on success.
result.html	Big score (e.g. 2/3), per-question pass/fail badges, Restart button that links to /.

5.3 jQuery pattern for quiz submission

Keep it simple — one handler that reads the selected answer, POSTs as JSON, redirects on response.

```
$('#submit-btn').on('click', function() {
  const answer = collectAnswer();  // reads checkboxes/radios based on question type
  $.ajax({
    url: '/submit-answer',
    method: 'POST',
    contentType: 'application/json',
    data: JSON.stringify({ question_id: QUESTION_ID, answer: answer }),
    success: function(res) { window.location.href = res.next_url; }
  });
});
```

Visual note from HW9 testing: Alita confused Forward Lean with Lumbar Flexion. Jolie, when you build the Lumbar Flexion lesson page, include the side-by-side comparison stick figures (currently only on Q2 answer slide) inline in the teaching section. Move the comparison visual earlier.

▸ **Claude Code prompt — Jolie — build base.html + home.html**

```
In templates/base.html, build a Bootstrap 5 scaffold (CDN, no npm). Include:
  – A green header bar (#2E5E4E) spanning full width, with the text 'SquatCheck' in
white, left-aligned.
  – A main {% block content %}{% endblock %} area, centered, max-width 900px, light gray
background (#F4F4F2).
  – A footer with {% block nav %}{% endblock %} for Back/Next buttons.
  – Link to /static/css/style.css and /static/js/app.js, plus jQuery CDN.

In templates/home.html, extend base.html. Content block: big centered 'SquatCheck' title
in dark green serif, subtitle 'Learn to spot compensation patterns in a squat', and a
large green 'Start Learning' button. The button posts to /start (use a small form with
method=POST, or a jQuery click handler that POSTs then redirects based on response).

Keep style.css minimal: the green (#2E5E4E), the light bg (#F4F4F2), and button styling.
Do not use external icon packs.
```

▸ **Claude Code prompt — Jolie — build learn.html (data-driven)**

```
In templates/learn.html, extend base.html. Expect a 'lesson' variable and 'total_lessons'
variable from Flask.

Show a section header with lesson.title. Then switch on lesson.type:
```

If lesson.type == 'intro': render lesson.body as a paragraph, then a numbered list where each item shows lesson.bullets[i].label in bold green + lesson.bullets[i].desc in plain text.

If lesson.type == 'comparison': render two Bootstrap cards side by side (col-md-6 each). Left card has a green header 'GOOD: {{ lesson.good.caption }}', shows /static/img/{{ lesson.good.image }}, caption below. Right card has a red header 'COMPENSATION: {{ lesson.bad.caption }}'. Below the two cards, show lesson.tip in a muted paragraph. If lesson has a 'side_by_side_note' field (lumbar flexion lesson), show it in a highlighted blue callout box.

In the {% block nav %}: a '< Back' link to /learn/{{ lesson.id - 1 }} (or / if lesson.id == 1), and a 'Next >' link to /learn/{{ lesson.id + 1 }} if lesson.id < total_lessons, else to /quiz/1.

Use Bootstrap classes, no custom JS needed here.

▸ Claude Code prompt — Jolie — build quiz.html (handles all 3 question types)

In templates/quiz.html, extend base.html. Expect a 'question' variable and 'total_questions' variable.

Section header: 'Quiz: Question {{ question.id }} of {{ total_questions }}'. Below it, show question.prompt.

Switch on question.type:

If 'multi_select': show /static/img/{{ question.image }} on the left, a 'Look for:' checklist box in the middle (hardcode the 3 checks: heels flat? torso vs shin angle? low back curved or neutral?), and on the right, checkboxes for each option.

If 'single_select': show question.notes in a styled observer-notes box, then radio buttons for each option.

If 'true_false_multi': loop through question.statements, render each as a row with the statement text on the left and two buttons (T / F) on the right, stored as a radio pair per statement.

Below, a green Submit button with id='submit-btn'. In a script tag, write a jQuery handler that:

1. Reads the answer based on question.type (a list of checked values for multi_select, a single string for single_select, a list of booleans for true_false_multi).
2. POSTs JSON { question_id: {{ question.id }}, answer: <collected> } to /submit-answer.
3. On success, redirects to response.next_url.

Pass question.type into the JS via a data attribute so the handler knows which collector to run.

▸ Claude Code prompt — Jolie — build result.html

In templates/result.html, extend base.html. Expect 'score' (int), 'total' (int), and 'breakdown' (list of {id, correct, prompt}).

```
Show a big centered score: '{{ score }} / {{ total }}' in dark green, 72px. Below it, a
message: 'Great job!' if score == total, 'Nice work!' if score >= total/2, else 'Keep
practicing!'.

Below the score, a list of the breakdown items: each row shows 'Question {{ item.id }}'
on the left, a green PASS or red FAIL badge on the right.

At the bottom, a 'Restart' button linking to /.
```

6. Testing & Click-Through — Sofia

6.1 Click-through checklist

Commit this as tests/clickthrough_checklist.md and run it fresh every time someone merges to main. A pass means the app is demo-ready.

1. Open / in a fresh browser window. Home page loads with Start button visible.
2. Click Start. URL changes to /learn/1. Intro content visible.
3. Click Next through all 5 lesson pages. URL advances /learn/1 → /learn/2 → ... → /learn/5.
4. Click Next from last lesson. URL changes to /quiz/1.
5. Q1: select two checkboxes, submit. Advances to /quiz/2 (or answer page if we add one).
6. Q2: pick a radio option, submit. Advances to /quiz/3.
7. Q3: mark four T/F statements, submit. Advances to /result.
8. Result page shows score out of 3 and per-question status.
9. Click Restart. Back at /. Session reset.
10. Back button in browser works at every stage (doesn't crash server).

6.2 Backend-data verification

This is the thing easy to forget — we need to confirm the server is actually storing selections, not just rendering pages.

- After clicking Start, check server log / printed state: start_time populated.
- After visiting each /learn/<n>, check lesson_visits has a timestamp for that n.
- After each quiz submit, check answers[<id>] has the answer and a correct boolean.
- On /result, verify displayed score matches sum of `correct` flags.

6.3 Bug reporting

File every issue on GitHub with: page URL, what you did, what happened, what you expected. Tag @sam for data bugs, @jolie for UI bugs. Don't fix Jolie's or Sam's code yourself unless they ask — confusion about who changed what kills group projects.

▸ Claude Code prompt — Sofia — write the click-through checklist file

```
In tests/clickthrough_checklist.md, write a markdown file I can fill in after every
merge. Include:
```

```
A 'How to run' section with the exact commands to start the app locally (pip install -r
requirements.txt, flask --app app run, open http://localhost:5000).


A 'Click-through' section with 10 numbered steps, each formatted as:
  [ ] 1. Action. Expected result. (Pass/Fail/Notes columns)


The 10 steps cover: home loads, Start advances to /learn/1, Next through all 5 lessons,
Next from last lesson goes to /quiz/1, Q1 multi-select submit advances, Q2 single-select
submit advances, Q3 T/F submit advances to /result, score displays correctly, Restart
returns to /, browser back button doesn't crash.


A 'Backend verification' section: after clicking Start, confirm start_time is printed in
server console; after each lesson visit, confirm lesson_visits has a new timestamp; after
each quiz submit, confirm answers[id] is populated with answer + correct flag; on
/result, confirm the score matches the sum of correct flags.


A 'Bug log' section at the bottom as a markdown table: Date | URL | What I did | What
happened | What I expected | Assigned to.


Keep it pasteable — no fancy formatting, just markdown checkboxes and tables so we can
fill it in during TA feedback.
```

7. Integration Plan — How We Avoid Stepping on Each Other

11. Day 1 (everyone): Sam pushes skeleton app.py with all routes returning placeholder strings. Jolie pushes base.html + empty child templates. Sofia writes the checklist.
12. Day 2: Sam fills in lessons.json and quiz.json with real content from the prototype. Jolie builds home.html and learn.html against real data.
13. Day 3: Sam wires /submit-answer and scoring. Jolie builds quiz.html with the jQuery handler. Sofia runs the checklist on main and files bugs.
14. Day 4: Fix bugs. Record the 1-minute YouTube walkthrough. Everyone pulls latest and confirms it runs on their own laptop (HW10 requirement).

Definition of done for HW10: all 10 checklist items pass, backend stores every selection, app runs on all three laptops, at least one commit per person on main, YouTube link recorded.

8. Open Questions to Resolve Before Coding

- Do we show a per-question answer page between quiz questions (like the prototype), or jump straight to the next question and show all answers on /result? Recommendation: skip per-question answer pages for HW10, add in HW11 for polish.
- Where do the stick-figure images come from — are we redrawing in SVG or exporting PNGs from the Google Slides? Recommendation: export PNGs for now, SVG in HW11.
- Jolie to confirm which screen the Lumbar vs Forward Lean side-by-side goes on (from Alita's feedback).